

Week 4

Networking & REST API

Pemrograman Mobile

Flutter • Dio • REST API • JSON

Dari data lokal → aplikasi yang berkomunikasi dengan backend

Learning Outcomes

Setelah mengikuti pertemuan ini, mahasiswa mampu:

1. Menjelaskan konsep HTTP, REST API, dan JSON.
2. Membuat model data dan serialization `fromJson` / `toJson` .
3. Mengimplementasikan request API menggunakan `Dio` .
4. Menerapkan repository pattern dasar untuk layer data.
5. Menangani loading, data, error, dan empty state.
6. Mengimplementasikan pagination dasar.
7. Memverifikasi kode networking yang dihasilkan AI.

Mengapa Networking?

Aplikasi Week 3 menyimpan data secara lokal.

Tanpa backend:
UI ↔ Provider ↔ Memory

Aplikasi nyata membutuhkan data dari server:

Dengan backend:
Flutter → Dio → REST API
→ Database → Kembali sebagai JSON

Tujuan: UI tidak bicara langsung ke internet, tetapi melalui repository.

Mini Poll

Jika API produk membutuhkan waktu 5 detik...

Apa yang harus dilihat user?

- A. Layar kosong tanpa info
- B. Aplikasi freeze / crash
- C. Loading indicator yang jelas
- D. Minta AI mempercepat internet

Diskusi

Apa masalah jika `http.get()` dipanggil langsung di `build()` ?

Bagian 1

HTTP & REST API

Apa Itu HTTP?

HTTP adalah protokol request-response.

```
Client (Flutter)
  | GET /products
  ↓
Server (REST API)
  | 200 OK + JSON
  ↓
Client menampilkan data
```

Method utama:

Method	Fungsi
GET	Membaca data
POST	Membuat data
PUT/PATCH	Mengubah data
DELETE	Menghapus data

Apa Itu REST API?

REST mengekspos resource lewat URL.

GET	/products	→ list produk
GET	/products/101	→ detail produk
POST	/products	→ tambah produk
PUT	/products/101	→ ubah produk
DELETE	/products/101	→ hapus produk

Response memakai status code:

200	OK
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Server Error

Frontend harus menangani tiap status, bukan hanya 200.

Apa Itu JSON?

JSON adalah format data teks yang dikirim API.

```
{  
  "id": 101,  
  "title": "Kopi Susu",  
  "price": 18000,  
  "inStock": true  
}
```

Di Dart menjadi `Map<String, dynamic>` :

```
final map = jsonDecode(responseBody);  
final title = map['title'] as String;
```

Aturan praktis:

- key bertipe `String`
- value bisa `String`, `num`, `bool`, `List`, `Map`, `null`
- selalu validasi tipe sebelum dipakai

Bagian 2

Model & Serialization

Mengapa Perlu Model?

Tanpa model, kode penuh string mentah:

```
Text(item['titel'].toString()) // typo = null
```

Dengan model, struktur eksplisit dan aman:

```
Text(product.title)
```

```
JSON (Map)  
  ↓ fromJson  
Product (Object)  
  ↓ toJson  
JSON (Map)
```

Prinsip: parsing JSON hanya terjadi di satu tempat di model.

Contoh Model Product

```
class Product {  
    final int id;  
    final String title;  
    final double price;  
  
    const Product({  
        required this.id,  
        required this.title,  
        required this.price,  
    });  
  
    factory Product.fromJson(  
        Map<String, dynamic> json,  
    ) {  
        return Product(  
            id: json['id'] as int,  
            title: json['title'] as String,  
            price: (json['price'] as num)  
                .toDouble(),  
        );  
    }  
}
```

Semua casting dan konversi ada di sini, bukan di UI.

Null Safety & Field Opsional

API nyata sering tidak konsisten.

```
factory Product.fromJson(  
    Map<String, dynamic> json,  
) {  
    return Product(  
        id: json['id'] as int? ?? 0,  
        title:  
            json['title'] as String? ??  
            'Tanpa nama',  
        price: (json['price'] as num?)  
            ?.toDouble() ?? 0.0,  
    );  
}
```

Aturan:

- gunakan `as String?`, bukan `as String` untuk field berisiko
- sediakan default value yang masuk akal
- jangan biarkan `null` merusak UI

Pertanyaan

Apa default terbaik jika `price` hilang: `0.0` atau error eksplisit?

Manual vs Codegen

Manual `fromJson`

- mudah dipahami
- cukup untuk model kecil

Codegen (`json_serializable` / `freezed`)

```
@JsonSerializable()  
class Product {  
  final int id;  
  final String title;  
}
```

- cocok untuk project besar
- mengurangi boilerplate
- butuh `build_runner`

Untuk Week 4, fokus utama:

Memahami alur parsing manual sebelum memakai generator.

Bagian 3

Dio & Request

Mengapa Dio?

`http` bawaan cukup, tetapi `Dio` lebih produktif.

<code>http</code>	<code>Dio</code>
<code>get()</code>	<code>+ baseUrl</code>
	<code>+ timeout</code>
	<code>+ interceptor</code>
	<code>+ cancel token</code>
	<code>+ FormData</code>

Keuntungan Dio:

- base URL terpusat & timeout global
- interceptor (auth, log)
- error bertipe `DioException`
- query parameter lebih rapi

Konfigurasi Dio

```
final dio = Dio(  
  BaseOptions(  
    baseUrl:  
      'https://api.example.com',  
    connectTimeout: const Duration(  
      seconds: 10,  
    ),  
    receiveTimeout: const Duration(  
      seconds: 10,  
    ),  
    headers: {  
      'Content-Type':  
        'application/json',  
    },  
  ),  
);
```

Perhatikan: semua request memakai konfigurasi yang sama.

GET: Ambil Daftar Produk

```
Future<List<Product>> fetchProducts()
    async {
        final response = await dio.get(
            '/products',
        );

        final list =
            response.data as List;

        return list
            .map(
                (e) => Product.fromJson(
                    e as Map<String, dynamic>,
                ),
            )
            .toList();
    }
```

Alur:

```
dio.get → response.data → List
      → fromJson → List<Product>
```

GET dengan Query & Path

Query parameter:

```
await dio.get(  
  '/products',  
  queryParameters: {  
    'category': 'food',  
    'page': 1,  
  },  
);
```

```
/products?category=food&page=1
```

Path parameter:

```
await dio.get('/products/101');
```

Jangan gabung string URL manual jika bisa memakai `queryParameters` .

Error Handling dengan Dio

Jangan hanya `try/catch` kosong.

```
try {  
  final res =  
    await dio.get('/products');  
  return parse(res.data);  
} on DioException catch (e) {  
  switch (e.type) {  
    case DioExceptionType  
      .connectionTimeout:  
    throw const NetworkFailure(  
      'Timeout. Periksa koneksi.',  
    );  
    case DioExceptionType.badResponse:  
    throw ServerFailure(  
      'Server: ${e.response?.statusCode}',  
    );  
    default:  
    throw const NetworkFailure(  
      'Gagal memuat data.',  
    );  
  }  
}
```

Ingat

Pesan error untuk user $\neq$ log teknis untuk developer.

Jenis Error yang Wajib Ditangani

No Internet	→ "Tidak ada koneksi"
Timeout	→ "Request timeout"
401 Unauthorized	→ "Sesi berakhir, login ulang"
404 Not Found	→ "Data tidak ditemukan"
500 Server Error	→ "Server bermasalah"
Parsing Error	→ "Format data salah"

Contoh status check:

```
if (e.response?.statusCode == 401) {  
  // refresh token / arahkan ke login  
}
```

Diskusi

Kapan retry otomatis cocok, dan kapan harus tombol "Coba lagi"?

Bagian 4

Repository Pattern Dasar

Apa Itu Repository?

Repository memisahkan UI dari sumber data.

```
UI (Widget)
  ↓ memanggil
Provider (Riverpod)
  ↓ memakai
Repository
  ↓ memakai
Dio / Data Source
  ↓
REST API
```

UI tidak tahu:

- base URL
- Dio
- JSON
- status code

UI hanya tahu:

```
Future<List<Product>> getProducts()
```

Contoh Repository

```
class ProductRepository {  
    final Dio dio;  
  
    ProductRepository(this.dio);  
  
    Future<List<Product>> getProducts()  
        async {  
        final response =  
            await dio.get('/products');  
  
        final list =  
            response.data['data'] as List;  
  
        return list  
            .map(  
                (e) => Product.fromJson(e),  
            )  
            .toList();  
    }  
}
```

Keuntungan menggunakan Repository:

- logic API terpusat
- mudah diganti data source
- mudah di-mock untuk testing

Contoh Repository + Riverpod

```
final dioProvider = Provider<Dio>(  
  (ref) => Dio(  
    BaseOptions(  
      baseUrl:  
        'https://api.example.com',  
    ),  
  ),  
);  
  
final productRepoProvider =  
  Provider<ProductRepository>(  
    (ref) => ProductRepository(  
      ref.watch(dioProvider),  
    ),  
);  
  
final productsProvider =  
  FutureProvider<List<Product>>(  
    (ref) => ref  
      .watch(productRepoProvider)  
      .getProducts(),  
  );
```

Widget → watch(productsProvider) → Repository → Dio → API

Bagian 5

UI State: Loading, Error, Empty

Empat State Wajib

Setiap layar yang menangani API harus punya 4 kondisi:

```
Loading → shimmer / spinner  
Data    → list produk  
Empty   → "Belum ada produk"  
Error    → pesan + tombol retry
```

Jangan biarkan user menebak:

- layar putih = bug atau loading?
- list kosong = error atau memang kosong?

Contoh AsyncValue.when

```
final state =  
    ref.watch(productsProvider);  
  
return state.when(  
    loading: () => const Center(  
        child:  
            CircularProgressIndicator(),  
    ),  
    data: (products) {  
        if (products.isEmpty) {  
            return const Center(  
                child: Text(  
                    'Belum ada produk.',  
                ),  
            );  
        }  
        return ProductList(products);  
    },  
    error: (e, _) => ErrorView(  
        message: '$e',  
        onRetry: () => ref.invalidate(  
            productsProvider,  
        ),  
    ),  
);
```

Pagination Dasar

API besar tidak dikirim sekaligus.

```
page=1 (10 item) → tampilkan  
scroll ke bawah  
page=2 (10 item) → tambah  
...
```

Konsep:

```
await dio.get(  
  '/products',  
  queryParameters: {  
    'page': page,  
    'limit': 10,  
  },  
);
```

State pagination:

```
initial → loading more → loaded  
          ↘ error
```

Week 4 cukup: pahami `page` + `limit` + append list.

Anti-Pattern

Hindari:

1. Panggil Dio di `build()`

```
build() → GET → rebuild → GET → loop
```

2. Parsing JSON di UI

Semua `json['...']` harusnya di model/repository.

3. `try/catch` yang menelan error

```
catch (e) {} // user tidak tahu apa-apa
```

4. Tidak ada timeout

Request menggantung selamanya saat server mati.

AI Lab

Gunakan AI sebagai Co-Developer

Contoh prompt:

Saya membuat aplikasi Flutter dengan Dio + Riverpod.

Endpoint: GET `https://api.example.com/products`

Response: `{ "data": [{ "id": 1, "title": "...", "price": 10000 }] }`

Buatkan:

1. Model Product + fromJson null-safe
2. ProductRepository dengan Dio
3. FutureProvider + UI dengan loading/error/empty/retry

Jelaskan alasan setiap bagian dan risikonya.

Jangan berhenti pada output AI.

AI Verification Challenge

Setelah AI menghasilkan kode:

1. **Baca** — pahami model, repository, provider.
2. **Verifikasi** — cek base URL, tipe data, null handling.
3. **Jalankan** — matikan internet, coba timeout, coba 404.
4. **Uji** — response kosong, field hilang, server error.
5. **Refactor** — pindahkan parsing ke model, rapikan error message.
6. **Jelaskan** — mampu menerangkan alur tanpa AI.

Uji minimal

- airplane mode
- field `price` = `null`
- `data` = `[]`
- status 500
- retry setelah error

Aktivitas Kelompok

Code Review 15 Menit

Cari minimal:

- 2 potensi bug parsing
- 2 masalah error handling
- 1 masalah arsitektur (logic API di UI)
- 1 masalah UX state
- 1 improvement dari kode AI

Kemudian jawab:

Apakah repository layer ini layak masuk production? Mengapa?

Quiz Cepat

1. Fungsi repository pattern?

- A. Menggambar UI
- B. Memisahkan UI dari sumber data
- C. Mengganti Flutter
- D. Mempercepat internet

2. `DioExceptionType.badResponse` artinya?

- A. Tidak ada error
- B. Server merespons dengan status error
- C. JSON selalu benar
- D. Aplikasi harus ditutup

3. Pagination memakai?

- A. `page` + `limit`
- B. Satu request raksasa
- C. Screenshot
- D. Hardcode semua data

Jawaban Quiz

1 → B

Repository memisahkan UI dari Dio/JSON/API.

2 → B

`badResponse` = server menjawab, tapi status menunjukkan error (mis. 404/500).

3 → A

```
?page=1&limit=10  
→ tampilkan → page++ → append
```

AI Challenge

Gunakan AI untuk membuat **versi awal**:

"Buatkan repository + provider + UI untuk aplikasi katalog produk dari REST API dummy dengan loading, empty, error, retry, dan pagination dasar."

Kemudian mahasiswa wajib:

1. memahami kode
2. menjalankan kode
3. menemukan minimal 2 kelemahan
4. memperbaiki error handling
5. menambahkan fitur sendiri (search/filter)
6. mencatat perubahan manual

Deliverables

Repository

```
week-04-networking-rest-api/  
├── lib/  
├── test/  
├── README.md  
└── screenshots/
```

README minimal berisi:

- deskripsi aplikasi & API yang dipakai
- model/repository/provider yang digunakan
- screenshot loading/data/empty/error
- cara menjalankan
- AI tools yang digunakan
- prompt utama
- perubahan manual
- masalah yang ditemukan dan hasil testing

Rubrik Week 4

Komponen	Bobot
Model & Serialization	20%
Dio & Repository	25%
State: Loading/Error/Empty	15%
UI & UX	10%
Testing	10%
Code Quality	10%
Responsible AI Usage	10%
Total	100%

Checklist Sebelum Submit

Data

- [] Model `fromJson` null-safe
- [] Repository memisahkan UI dari Dio
- [] Timeout dikonfigurasi
- [] Pagination dasar dipahami

UI

- [] Loading / Data / Empty / Error tampil benar
- [] Tombol retry berfungsi
- [] Tidak ada request di `build()`

AI

- [] Prompt dicatat & Output AI diverifikasi
- [] Ada perubahan manual
- [] Kode dipahami mahasiswa

Exit Ticket

Jawab sebelum kelas berakhir:

1. Apa perbedaan:

http
vs
Dio

2. Apa fungsi:

Model
vs
Repository

3. Sebutkan 4 state wajib layar menangani API beserta contoh UI-nya.

4. Mengapa error handling AI tetap harus diuji manual?

Key Takeaways

Networking

Dio membuat request HTTP lebih terstruktur.

Arsitektur

Repository memisahkan UI dari sumber data.

UI State

Loading/Data/Empty/Error harus eksplisit.

AI

AI merancang repository awal.

Developer menguji error handling dan memperbaikinya.

Next Week

Week 5: Local Storage & Offline First

```
REST API  
↓  
Repository  
↓  
Cache (Hive / SQLite)  
↓  
UI tetap jalan saat offline
```

Target

Aplikasi tetap berguna tanpa internet melalui caching dan sync dasar.

Build → Verify → Explain

Prinsip Pembelajaran Mobile



Jangan hanya bisa memanggil API.

Jadilah developer yang memahami alur data yang dipanggil.